



20. Valid Parentheses

Description:

- ✓ Given a string `s` containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.
- ✓ An input string is valid if:
 - Open brackets must be closed by the same type of brackets.
 - Open brackets must be closed in the correct order.
 - Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: `str = "()`

Output: `true`

Example 2:

Input: `str = "()[]{}"`

Output: `true`

Example 3:

Input: `str = "["`

Output: `false`

Constraints:

- a) $1 \leq s.length \leq 104$
- b) `s` consists of parentheses only '`()[]{}'`.



Java Learning Center

No.1 In Java Training & placement

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 9

Bloomberg 5

Google 5

Apple 3

Microsoft 2

Adobe 2

Yahoo 2

JPMorgan 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

BlackRock 18

Facebook 16

Spotify 9

LinkedIn 8

Uber 6

Zillow 6

Yandex 5

Oracle 5

Cisco 3

IBM 3

Netflix 3

tcs 3

Mathworks 2

eBay 2

VMware 2

Qualcomm 2

Lyft 2

Goldman Sachs 2

Walmart Global Tech 2

Intel 2

SAP 2

Twitch 2

TikTok 2

Société Générale 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Expedia 16

Bolt 8

ServiceNow 6

Salesforce 6

Paypal 4

Morgan Stanley 4

Atlassian 3

Booking.com 3

Nvidia 3

Splunk 3

Arista Networks 3

Twilio 2

Tesla 2

Barclays 2

Twitter 2

Siemens 2

Visa 2

Huawei 2

Intuit 2

Dataminr 2

Infosys 2

Accolite 2

Nagarro 2

MakeMyTrip 2

Airbnb

Zenefits

20. Valid Parentheses - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```



```
class Solution {

    private Map<Character, Character> map ;

    public Solution() {
        this.map = new HashMap<Character, Character>();
        this.map.put('(', ')');
        this.map.put('{', '}');
        this.map.put('[', ']');
    }

    public boolean isValid(String str) {

        Deque<Character> mystack = new ArrayDeque<>();

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);

            if (map.containsKey(ch)) {
                mystack.push(ch);
            } else {
                if(mystack.isEmpty()) {
                    return false;
                }
                char open = mystack.pop();
                if ( map.get(open)!=ch) {
                    return false;
                }
            }
        }
        } // for end

        return mystack.isEmpty();
    }
}
```

155. Min Stack

Description:

- ✓ Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.
- ✓ **Implement the MinStack class:**
 - a) **MinStack()**: initializes the stack object.
 - b) **void push(int val)**: pushes the element val onto the stack.
 - c) **void pop()**: removes the element on the top of the stack.
 - d) **int top()**: gets the top element of the stack.
 - e) **int getMin()**: retrieves the minimum element in the stack.
- ✓ You must implement a solution with $O(1)$ time complexity for each function.

Constraints:

- a) $-2^{31} \leq \text{val} \leq 2^{31} - 1$
- b) Methods pop, top and getMin operations will always be called on non-empty stacks.
- c) At most $3 * 10^4$ calls will be made to push, pop, top, and getMin.

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Expedia 3

Amazon 2

Bloomberg 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Microsoft 6

Apple 3

Adobe 2



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Facebook 9

VMware 5

Google 4

Salesforce 4

Oracle 4

Walmart Global Tech 3

Goldman Sachs 3

Yahoo 2

Airtel 2

Arcesium 2

Uber

Snapchat

Zenefits

155. Min Stack - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MinStack {
    Deque<int[]> mystack = new ArrayDeque<>();

    public void push(int element) {
        if (mystack.isEmpty()) {
            mystack.push(new int[]{element,element});
            return;
        }
        int currMin = mystack.peek()[1];
        mystack.push(new int[]{element, Math.min(element, currMin)});
    }
    public void pop() {
        mystack.pop();
    }
    public int top() {
        return mystack.peek()[0];
    }
    public int getMin() {
        return mystack.peek()[1];
    }
}
```



1472. Design Browser History

Description:

- ✓ You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

- ✓ **Implement the BrowserHistory class:**

a) BrowserHistory(string homepage) :

Initializes the object with the homepage of the browser.

b) void visit(string url) :

Visits url from the current page. It clears up all the forward history.

c) string back(int steps) :

Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history at most steps.

d) string forward(int steps) :

Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history at most steps.

Constraints:

- a) $1 \leq \text{homepage.length} \leq 20$
- b) $1 \leq \text{url.length} \leq 20$
- c) $1 \leq \text{steps} \leq 100$
- d) homepage and url consist of '.' or lower case English letters.
- e) At most 5000 calls will be made to visit, back, and forward.

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Bloomberg 4

Roblox 4

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon 6

DoorDash 2

Oracle 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Google 4

Microsoft 2

1472. Design Browser History

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class BrowserHistory {

    Deque<String> backHistory, forwardHistory;
    String current;

    public BrowserHistory(String homepage) {
        backHistory = new ArrayDeque<>();
        forwardHistory = new ArrayDeque<String>();
        current = homepage;
    }
}
```



```
public void visit(String url) {
    backHistory.push(current);
    current = url;
    forwardHistory = new ArrayDeque<String>();
}

public String back(int steps) {
    while(steps > 0 && !backHistory.isEmpty()) {
        forwardHistory.push(current);
        current = backHistory.pop();
        steps--;
    }
    return current;
}

public String forward(int steps) {
    while(steps > 0 && !forwardHistory.isEmpty()) {
        backHistory.push(current);
        current = forwardHistory.pop();
        steps--;
    }
    return current;
}
}
```



71. Simplify Path

Description:

- ✓ Given a string path, which is an absolute path (starting with a slash '/') to a file or directory in a Unix-style file system, convert it to the simplified canonical path.
- ✓ In a Unix-style file system, a period '.' refers to the current directory, a double period '..' refers to the directory up a level, and any multiple consecutive slashes (i.e. '//') are treated as a single slash '/'. For this problem, any other format of periods such as '...' are treated as file/directory names.
- ✓ The canonical path should have the following format:
 - The path starts with a single slash '/'.
 - Any two directories are separated by a single slash '/'.
 - The path does not end with a trailing '/'.
 - The path only contains the directories on the path from the root directory to the target file or directory (i.e., no period '.' or double period '..')
- ✓ Return the simplified canonical path.

Example 1:

Input: path = "/home/" **Output:** "/home"

Example 2:

Input: path = "/../" **Output:** "/"

Example 3:

Input: path = "/home//foo/" **Output:** "/home/foo"

Constraints:

- a) $1 \leq \text{path.length} \leq 3000$
- b) path consists of English letters, digits, period '.', slash '/' or '_'.
- c) path is a valid absolute Unix path.

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Facebook 2

Yandex 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon 5

Microsoft 3

Google 3

Uber 2

Oracle 2

tcs 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Apple 3

Wish 3

Bloomberg 2

VMware 2

Pinterest 2



71. Simplify Path- Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public String simplifyPath(String path) {

        Stack<String> mystack = new Stack<>();
        String[] directoryList = path.split("/");

        for (String directory : directoryList) {
            if (directory.equals(".") || directory.isEmpty()) {
                continue;
            } else if (directory.equals("..")) {
                if (!mystack.isEmpty()) {
                    mystack.pop();
                }
            } else {
                mystack.add(directory);
            }
        }

        StringBuilder result = new StringBuilder();
        for (String dir : mystack) {
            result.append("/");
            result.append(dir);
        }

        return result.length() > 0 ? result.toString() : "/" ;
    }
}
```



394. Decode String

Description:

- ✓ Given an encoded string, return its decoded string.
- ✓ The encoding rule is: k[encoded_string], where the encoded_string inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer.
- ✓ You may assume that the input string is always valid;

Example 1:

Input: s = "3[a]2[bc]"

Output: "aaabcbc"

Example 2:

Input: s = "3[a2[c]]"

Output: "accaccacc"

Example 3:

Input: s = "2[abc]3[cd]ef"

Output: "abcbcccdcdcdcf"

Constraints:

- 1 <= s.length <= 30
- s consists of lowercase English letters, digits, and square brackets '['].
- s is guaranteed to be a valid input.
- All the integers in s are in the range [1, 300].

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Bloomberg 14 Google 4 TikTok 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon 14 Microsoft 9 Apple 6 Adobe 6 Uber 6 C3 IoT 6

ByteDance 4 Yahoo 3 Salesforce 2 eBay 2 Qualtrics 2

Square 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Cisco 17 Snapchat 15 Facebook 13 Oracle 9 Intuit 4

Sumologic 4 Cruise Automation 3 Walmart Global Tech 3 HBO 3

SAP 2 Goldman Sachs 2 ServiceNow 2 Jump Trading 2

Millennium 2 Coupang Yelp

394. Decode String - Solution

```

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {

```



```
String decodeString(String str) {

    Stack<Integer> digitStack = new Stack<>();
    Stack<StringBuilder> resultStack = new Stack<>();

    StringBuilder resultString = new StringBuilder();
    int k = 0;

    for (char ch : str.toCharArray()) {

        if (Character.isDigit(ch)) {
            k = k * 10 + ch - '0';
        } else if (ch == '[') {
            digitStack.push(k);
            resultStack.push(resultString);
            resultString = new StringBuilder();
            k = 0;
        } else if (ch == ']') {

            StringBuilder decodedString = resultStack.pop();
            int digit = digitStack.pop();
            for (int i=1;i<=digit;i++) {
                decodedString.append(resultString);
            }

            resultString = decodedString;

        } else {
            resultString.append(ch);
        }
    }
    return resultString.toString();
}
```